Facultad de Ingeniería
Departamento de Ciencias Básicas
Área de Programación
Programación Avanzada
Ing. Jorge Estuardo Tello Monzón

Universidad Rafael Landívar
Tradición Jesuita en Guatemala

Proyecto de aplicación 02
XelaTech Soporte

Rudy Isaac Cardona Santisteban
Carnet: 1580226

Guatemala, 12 de octubre del 2026

1. Descripción de entradas, procesos y salidas

- **1. Clientes**
  Entradas: Nombre completo, teléfono, correo, tipo de cliente; código para consultar o actualizar.
  Procesos: Validar campos; generar código; agregar a la lista; consultar por código; actualizar teléfono y correo.
  Salidas: Cliente registrado con su código; listado y detalle; mensajes de confirmación o error.
- **2. Técnicos**
  Entradas: Nombre completo, especialidad, estado de disponibilidad.
  Procesos: Validar; generar código; asignar fila de la matriz; calcular promedio de calificaciones; listar por especialidad.
  Salidas: Listado de técnicos con carga y promedio; técnicos por especialidad; distribución de calificaciones (arreglo).
- **3. Equipos**
  Entradas: Tipo de equipo, número de serie, marca, modelo, código del propietario, estado físico, accesorios y dato propio del tipo.
  Procesos: Comprobar que el propietario existe y que la serie no esté duplicada; crear la subclase correspondiente; generar código interno.
  Salidas: Equipo registrado con su código; listado de equipos.
- **4. Solicitudes**
  Entradas: Código de cliente y de equipo, descripción, prioridad; diagnóstico y costo adicional; código de repuesto y cantidad; nuevo estado; calificación.
  Procesos: Validar existencia; impedir segunda solicitud activa del equipo; crear y encolar; registrar diagnóstico; descontar repuestos; validar y apilar cambios de estado; deshacer; calcular total; entregar o cancelar.
  Salidas: Solicitud con código; historial de estados; costos y total; alertas de repuestos; mensajes de error.
- **5. Repuestos**
  Entradas: Nombre, categoría, existencia, existencia mínima, costo unitario, precio de venta; cantidad a agregar.
  Procesos: Validar datos no negativos y precio ≥ costo; generar código; sumar existencia; comparar con el mínimo.
  Salidas: Repuesto registrado; listado con situación (normal, mínimo, agotado).
- **6. Atención de colas**
  Entradas: Opción de atender la siguiente solicitud o mostrar colas y matriz.
  Procesos: Elegir cola (urgente primero, FIFO); buscar técnico apto; desencolar; asignar; registrar en la matriz.
  Salidas: Solicitud atendida y técnico asignado; estado de ambas colas; matriz de carga con técnico y día de mayor carga.
- **7. Ordenamientos**
  Entradas: Opción de ordenamiento y sentido (ascendente o descendente).
  Procesos: Verificar al menos 2 datos; copiar la lista; ordenar con el algoritmo manual correspondiente.
  Salidas: Reporte con algoritmo, criterio, sentido y datos ordenados.
- **8. Búsquedas**
  Entradas: Texto de nombre, estado, marca o código de solicitud.
  Procesos: Recorrido secuencial con todas las coincidencias; búsqueda binaria previa verificación de orden; guardar el resultado.
  Salidas: Lista de coincidencias o aviso de que no hay resultados.
- **9. Reportes**
  Entradas: Número de reporte.
  Procesos: Recorrer listas, colas y arreglos; contar, sumar y ordenar según el reporte.
  Salidas: Los 12 reportes mínimos.
- **10. Datos de prueba**
  Entradas: Cantidad de registros a generar.
  Procesos: Generar con Random clientes, técnicos, repuestos, equipos y solicitudes; códigos únicos; aplicar las mismas validaciones.
  Salidas: Resumen de registros generados.

- **11. Salir**
  Entradas: Confirmación.
  Procesos: Terminar el ciclo del menú.
  Salidas: Cierre del programa.

2. Clases, atributos, métodos y relaciones

Convención de visibilidad: `#` protegido, `-` privado, `+` público. Los atributos privados se exponen mediante propiedades con validación en el `set`, de modo que ningún objeto quede con datos inválidos.

2.1 Enumeraciones

- **EstadoSolicitud** — Recibida, EnEspera, Asignada, EnDiagnostico, EnReparacion, ListaParaEntregar, Entregada, Cancelada. **Uso:** Estado de la solicitud y datos del historial.
- **PrioridadSolicitud** — Normal, Urgente. **Uso:** Decide en qué cola se coloca la solicitud.
- **TipoCliente** — Particular, Estudiante, Profesional, Negocio, Institucion. **Uso:** Clasificación del cliente.
- **Especialidad** — Portatiles, Escritorio, Impresoras. **Uso:** Especialidad del técnico y requisito del equipo.
- **TipoEquipo** — ComputadoraPortatil, ComputadoraEscritorio, Impresora. **Uso:** Identifica el tipo; indexa el arreglo de equipos por tipo.

2.2 Clases del modelo

Persona (abstracta)
base de Cliente y Tecnico

Atributos: # codigo : string; # nombreCompleto : string

Métodos:

- + Persona(codigo, nombre) — valida campos obligatorios
- + ObtenerResumen() : string — abstracto
- + ToString() : string — sobrescrito

Cliente : Persona
persona que solicita el servicio

Atributos: - telefono : string; - correo : string; - tipoCliente : TipoCliente; - cantidadSolicitudes : int

Métodos:

- + Cliente(codigo, nombre, telefono, correo, tipo) — constructor completo
- + Cliente(codigo, nombre, telefono, correo) — sobrecarga; tipo = Particular
- + ActualizarContacto(telefono, correo) — valida y actualiza
- + IncrementarSolicitudes() — suma 1 al crear una solicitud
- + ObtenerResumen() : string — sobrescrito

Tecnico : Persona
personal que repara los equipos

Atributos: + LIMITE_TRABAJOS : int = 3 (constante); - especialidad : Especialidad; - disponible : bool; - trabajosActivos : int; - serviciosFinalizados : int; - sumaCalificaciones : int; - cantidadCalificaciones : int; - indiceMatriz : int (fila fija)

Métodos:

- + PuedeRecibir(esp : Especialidad) : bool — especialidad, disponible y trabajosActivos < LIMITE
- + IniciarTrabajo() — trabajosActivos++ (valida límite)

- + TerminarTrabajo() — trabajosActivos--
- + ReanudarTrabajo() — usado al deshacer; valida límite
- + RegistrarServicioFinalizado(calificacion) — finalizados++, acumula calificación
- + CalcularPromedio() : double — suma / cantidad (0 si no hay)
- + CambiarDisponibilidad(valor : bool)
- + ObtenerResumen() : string — sobrescrito

Equipo (abstracta)
base de los tipos de equipo

Atributos: # codigoInterno : string; # numeroSerie : string; # marca : string; # modelo : string; # propietario : Cliente; # estadoFisico : string; # accesorios : string

Métodos:

- + Equipo(codigo, serie, marca, modelo, propietario, estadoFisico, accesorios)
- + ObtenerTipo() : TipoEquipo — abstracto
- + ObtenerEspecialidadRequerida() : Especialidad — abstracto
- + CalcularCostoRevision() : decimal — abstracto
- + EstimarTiempoHoras() : double — abstracto
- + GenerarDiagnosticoSugerido() : string — virtual
- + ToString() : string — sobrescrito

ComputadoraPortatil : Equipo
regla particular: batería

Atributos: - requiereRevisionBateria : bool

Métodos:

- Sobrescribe los cuatro métodos abstractos y `GenerarDiagnosticoSugerido()`
- + constructor propio que recibe `requiereRevisionBateria`

ComputadoraEscritorio : Equipo
regla particular: monitor

Atributos: - incluyeMonitor : bool

Métodos:

- Sobrescribe los cuatro métodos abstractos y `GenerarDiagnosticoSugerido()`
- + constructor propio que recibe `incluyeMonitor`

Impresora : Equipo
regla particular: tipo de impresión

Atributos: - tipoImpresion : string (Tinta | Láser)

Métodos:

- Sobrescribe los cuatro métodos abstractos y `GenerarDiagnosticoSugerido()`
- + constructor propio que recibe `tipoImpresion`

Solicitud
orden de servicio; centro del sistema

Atributos: - codigo : string; - cliente : Cliente; - equipo : Equipo; - descripcion : string; - fechaRecepcion : DateTime; - prioridad : PrioridadSolicitud; - estado : EstadoSolicitud; - tecnico : Tecnico (nulo hasta asignar); - diagnostico : string; - costoManoObra : decimal; - tiempoEstimadoHoras : double; - fechaEntrega : DateTime?; - calificacion : int; - repuestosUtilizados : List<RepuestoUtilizado>; - historial : Stack<RegistroEstado>; (calculados) CostoRepuestos, Total

Métodos:

- + Solicitud(codigo, cliente, equipo, descripcion, prioridad)
- + Solicitud(codigo, cliente, equipo, descripcion) — sobrecarga; prioridad = Normal
- + EsTransicionValida(nuevo) : bool — tabla de transiciones
- + CambiarEstado(nuevo, obs) — valida, apila el estado anterior y cambia
- + DeshacerUltimoCambio() : EstadoSolicitud — valida pila no vacía; desapila
- + ConsultarHistorial() : List<RegistroEstado> — recorre la pila sin desapilar
- + AsignarTecnico(t) / RegistrarDiagnostico(texto, costoManoObra)
- + AgregarRepuesto(r, cantidad) y AgregarRepuesto(r) — sobrecarga (cantidad = 1)
- + CalcularCostoRepuestos() : decimal / CalcularTotal() : decimal
- + EstaActiva() : bool / EstaFinalizada() : bool

RepuestoUtilizado
línea de repuesto dentro de una solicitud

Atributos: - repuesto : Repuesto; - cantidad : int; - precioAplicado : decimal

Métodos:

- + CalcularSubtotal() : decimal — cantidad × precioAplicado

Repuesto
pieza del inventario

Atributos: - codigo : string; - nombre : string; - categoria : string; - existencia : int; - existenciaMinima : int; - costoUnitario : decimal; - precioVenta : decimal; - cantidadUtilizada : int

Métodos:

- + HayExistencia(cantidad) : bool
- + Descontar(cantidad) — valida existencia; suma a cantidadUtilizada
- + AgregarExistencia(cantidad)
- + EstaAgotado() : bool — existencia = 0
- + EstaEnMinimo() : bool — existencia ≤ existenciaMinima

RegistroEstado
elemento de la pila de historial

Atributos: - estadoAnterior : EstadoSolicitud; - fechaHora : DateTime; - observacion : string

Métodos:

- Solo propiedades de lectura y constructor

2.3 Polimorfismo en la jerarquía de equipos

- **ObtenerTipo()**
  ComputadoraPortatil: ComputadoraPortatil
  ComputadoraEscritorio: ComputadoraEscritorio
  Impresora: Impresora
- **ObtenerEspecialidadRequerida()**
  ComputadoraPortatil: Portatiles
  ComputadoraEscritorio: Escritorio
  Impresora: Impresoras
- **CalcularCostoRevision()**
  ComputadoraPortatil: Q120 (+Q30 si requiere revisión de batería)
  ComputadoraEscritorio: Q90 (+Q20 si incluye monitor)
  Impresora: Q70 (+Q25 si es láser)

- **EstimarTiempoHoras()**
  ComputadoraPortatil: 3 h (+1 h batería)
  ComputadoraEscritorio: 2 h (+1 h monitor)
  Impresora: 1.5 h (+1 h láser)
- **GenerarDiagnosticoSugerido()**
  ComputadoraPortatil: Revisar batería, disco, memoria, pantalla y ventilación.
  ComputadoraEscritorio: Revisar fuente de poder, disco, memoria y tarjeta madre.
  Impresora: Revisar cabezal o tóner, rodillos, alimentador y conexión.

2.4 Clases de servicio y control

SistemaXelaTech

contexto central que agrupa los módulos

Atributos: + Clientes : GestorClientes; + Tecnicos : GestorTecnicos; + Equipos : GestorEquipos; + Repuestos : GestorRepuestos; + Solicitudes : GestorSolicitudes; + Carga : MatrizCarga; + Busquedas : List<ResultadoBusqueda>

Métodos:

- + SistemaXelaTech() — crea los gestores e inicializa estructuras

GestorClientes

administra `List<Cliente>`

Atributos: - clientes : List<Cliente>; - contador : int

Métodos:

- + Registrar(...) : Cliente — valida y genera código
- + Actualizar(codigo, telefono, correo)
- + ObtenerPorCodigo(codigo) : Cliente
- + ObtenerTodos() : List<Cliente>
- - GenerarCodigo() : string

GestorTecnicos

administra `List<Tecnico>` y el arreglo de calificaciones

Atributos: - tecnicos : List<Tecnico>; - calificaciones : int[5] (índice = calificación − 1); - contador : int

Métodos:

- + Registrar(nombre, especialidad) : Tecnico — máximo 10
- + ObtenerPorCodigo(codigo) : Tecnico
- + BuscarMejorDisponible(esp) : Tecnico — menor carga
- + RegistrarCalificacion(valor) — incrementa el arreglo
- + ObtenerDistribucion() : int[]
- + ObtenerTodos() : List<Tecnico>

GestorEquipos

administra `List<Equipo>`

Atributos: - equipos : List<Equipo>; - contador : int

Métodos:

- + Registrar(equipo) — rechaza serie duplicada
- + ExisteSerie(serie) : bool
- + ObtenerPorCodigo(codigo) : Equipo
- + ObtenerTodos() : List<Equipo>

GestorRepuestos
administra `List<Repuesto>`

Atributos: - repuestos : List<Repuesto>; - contador : int

Métodos:

- + Registrar(...) : Repuesto
- + ObtenerPorCodigo(codigo) : Repuesto
- + AgregarExistencia(codigo, cantidad)
- + ObtenerTodos() : List<Repuesto>

GestorSolicitudes
lista maestra, colas y reglas del flujo

Atributos: - solicitudes : List<Solicitud>; - colaUrgente : Queue<Solicitud>; - colaNormal : Queue<Solicitud>; - contador : int

Métodos:

- + CrearSolicitud(codCliente, codEquipo, descripcion, prioridad) : Solicitud
- + AtenderSiguiente() : Solicitud
- + RegistrarDiagnostico(codigo, texto, adicional)
- + AgregarRepuesto(codigo, codRepuesto, cantidad)
- + ActualizarEstado(codigo, nuevo, obs) — incluye cancelar
- + RegistrarEntrega(codigo, calificacion)
- + DeshacerUltimoCambio(codigo) / ConsultarHistorial(codigo)
- + TieneSolicitudActiva(codEquipo) : bool
- + ObtenerPorCodigo(codigo) / ObtenerEnCola(prioridad) / ObtenerTodas()

MatrizCarga
carga semanal de trabajo

Atributos: + MAX_TECNICOS : int = 10 (constante); + DIAS : int = 7 (constante); - carga : int[10, 7]

Métodos:

- + RegistrarAsignacion(fila, dia : DayOfWeek)
- + Mostrar(tecnicos)
- + ObtenerFilaMayorCarga() : int — mayor suma por fila
- + ObtenerDiaMayorCarga() : int — mayor suma por columna

Ordenador (estática)
algoritmos de ordenamiento manuales

Métodos:

- + BubbleSort<T>(lista, clave, asc) / SelectionSort<T>(...) / ShellSort<T>(...)
- + QuickSort<T>(lista, bajo, alto, clave, asc)
- - FueraDeOrden<T>(a, b, clave, asc) : bool / Intercambiar<T>(lista, i, j)

Buscador (estática)
algoritmos de búsqueda manuales

Métodos:

- + ClientesPorNombre(lista, texto) : List<Cliente>
- + SolicitudesPorEstado(lista, estado) : List<Solicitud>
- + EquiposPorMarca(lista, marca) : List<Equipo>

- + BinariaPorCodigo(lista, codigo) : Solicitud
- + EstaOrdenadaPorCodigo(lista) : bool

ResultadoBusqueda
bitácora de una búsqueda ejecutada

Atributos: - tipoBusqueda : string; - criterio : string; - valorBuscado : string; - coincidencias : List<string>; - fechaHora : DateTime

Métodos:

- Propiedades de lectura; alimenta el reporte 11

Reportes
los 12 reportes mínimos

Atributos: - sistema : SistemaXelaTech

Métodos:

- + EsperaPorPrioridad(), ActivasFinalizadasCanceladas(), TecnicosDisponiblesYCompletos(), TecnicoConMasFinalizados()
- + EquiposPorTipo(), SolicitudesPorCosto(), SolicitudesPorFechaOTiempo(), RepuestosPorExistencia()
- + RepuestosMinimosOAgotados(), IngresosTotales(), ResultadosBusquedas(), MatrizSemanal()

GeneradorDatos
datos de prueba con Random

Atributos: - aleatorio : Random; - sistema : SistemaXelaTech

Métodos:

- + Generar(cantidad)
- - GenerarClientes(n), GenerarTecnicos(n), GenerarRepuestos(n), GenerarEquiposYSolicitudes(n)

Validador (estática)
validaciones de entrada reutilizables

Métodos:

- + TextoObligatorio(campo, valor), NoNegativo(campo, valor), EnteroPositivo(campo, valor)
- + CalificacionValida(valor), TelefonoValido(valor), CorreoValido(valor)

ReglaNegocioException : Exception
error por incumplir una regla del proceso

Métodos:

- + ReglaNegocioException(mensaje)

Consola (estática)
lectura segura de datos

Métodos:

- + LeerEntero(msg, min, max) : int — repite hasta recibir un valor válido
- + LeerDecimal(msg), LeerTexto(msg), Confirmar(msg) : bool, Pausa()

MenuConsola / Program
interfaz y punto de entrada

Atributos: - sistema : SistemaXelaTech

Métodos:

- + Iniciar() — ciclo del menú principal con `try/catch`
- - MenuClientes(), MenuTecnicos(), MenuEquipos(), MenuSolicitudes(), MenuRepuestos(),
- MenuColas(), MenuOrdenamientos(), MenuBusquedas(), MenuReportes(), MenuDatosPrueba()
- Program.Main() solo crea `MenuConsola` y llama `Iniciar()`

2.5 Relaciones entre clases

- **Cliente, Tecnico** — Herencia — **Persona** (—): Comparten código y nombre.
- **ComputadoraPortatil, ComputadoraEscritorio, Impresora** — Herencia — **Equipo** (—): Tipos de equipo con reglas propias.
- **Equipo** — Asociación — **Cliente** (0..* → 1): Propietario del equipo.
- **Solicitud** — Asociación — **Cliente** (0..* → 1): Cliente que solicita el servicio.
- **Solicitud** — Asociación — **Equipo** (0..* → 1): Equipo en servicio (solo una solicitud activa a la vez).
- **Solicitud** — Asociación — **Tecnico** (0..* → 0..1): Técnico asignado (nulo hasta la asignación).
- **Solicitud** — Composición — **RepuestoUtilizado** (1 → 0..*): Repuestos usados; no existen sin la solicitud.
- **RepuestoUtilizado** — Asociación — **Repuesto** (* → 1): Repuesto del inventario que se consumió.
- **Solicitud** — Composición — **RegistroEstado** (1 → 0..*): Pila de estados anteriores.
- **SistemaXelaTech** — Composición — **Gestores, MatrizCarga, Reportes, GeneradorDatos** (1 → 1): Agrupa los módulos.
- **Gestores** — Agregación — **Entidades del modelo** (1 → 0..*): Cada gestor guarda su lista de objetos.
- **GestorSolicitudes** — Dependencia — **Otros gestores, MatrizCarga** (—): Consulta clientes, equipos, técnicos y repuestos.
- **Reportes** — Dependencia — **Ordenador, MatrizCarga** (—): Reportes ordenados y matriz.
- **MenuConsola** — Asociación — **SistemaXelaTech** (1 → 1): El menú opera sobre el sistema.

2.6 Conceptos de POO aplicados

- **Encapsulamiento:** Atributos protegidos en `Persona` y `Equipo`; atributos privados con propiedades validadas; por ejemplo, `trabajosActivos` solo cambia mediante `IniciarTrabajo`, `TerminarTrabajo` y `ReanudarTrabajo`.
- **Herencia:** `Persona` → `Cliente`, `Tecnico`; `Equipo` → `ComputadoraPortatil`, `ComputadoraEscritorio`, `Impresora`.
- **Polimorfismo y sobrescritura:** Métodos abstractos y virtual de `Equipo`; `ObtenerResumen()` y `ToString()`. Las listas de `Equipo` se tratan de forma uniforme.
- **Sobrecarga:** Constructores de `Cliente` y de `Solicitud`; método `AgregarRepuesto(Repuesto, int)` y `AgregarRepuesto(Repuesto)`.
- **Relaciones:** Sección 4.5.
- **Excepciones y validación:** `ReglaNegocioException`, clase `Validador`, lectura con `TryParse` y bloque `try/catch` en el ciclo del menú.

3. Diagramas de flujo del algoritmo principal

Figura 1. Flujo del programa principal: inicialización, ciclo del menú y control de errores.

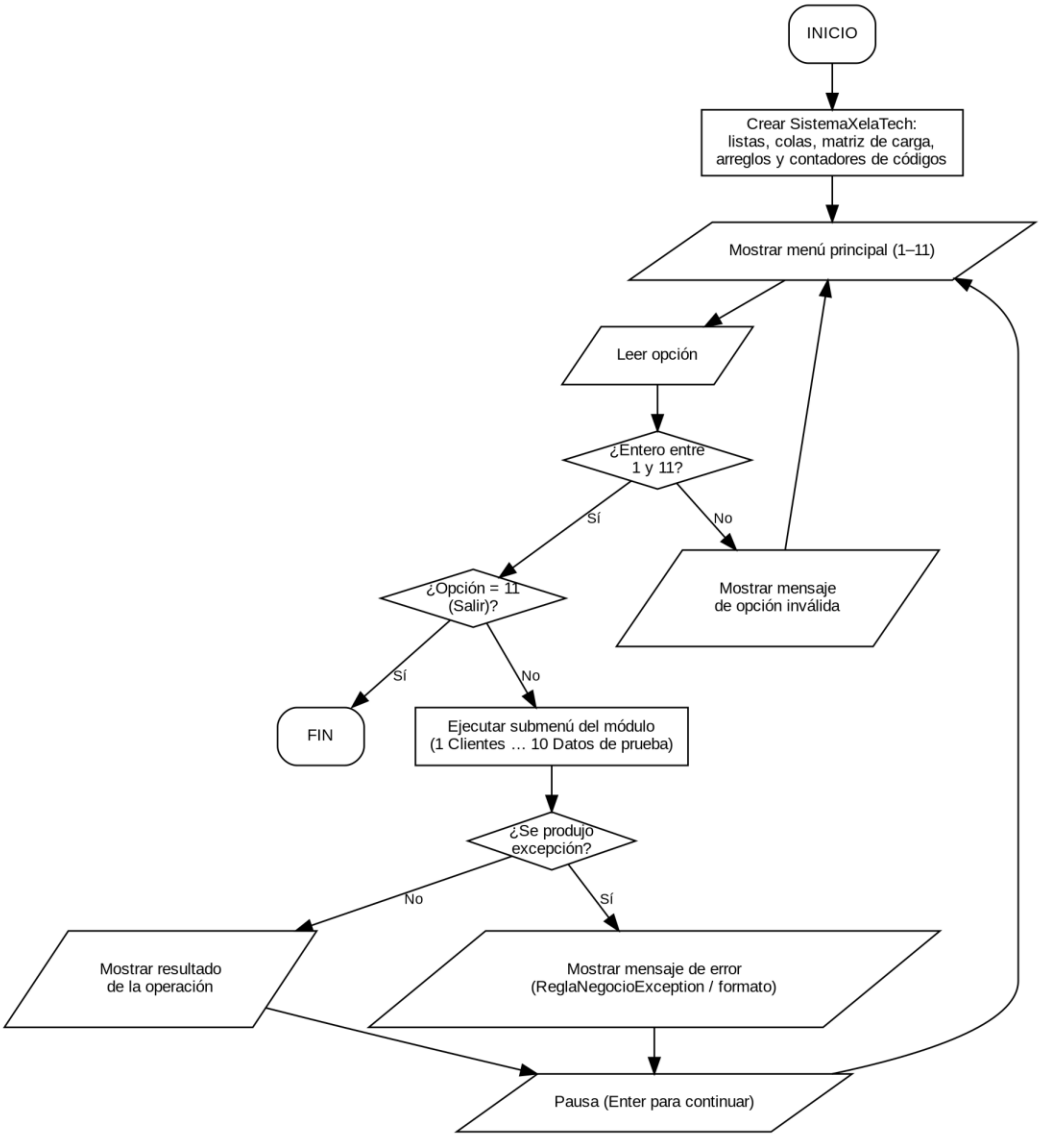
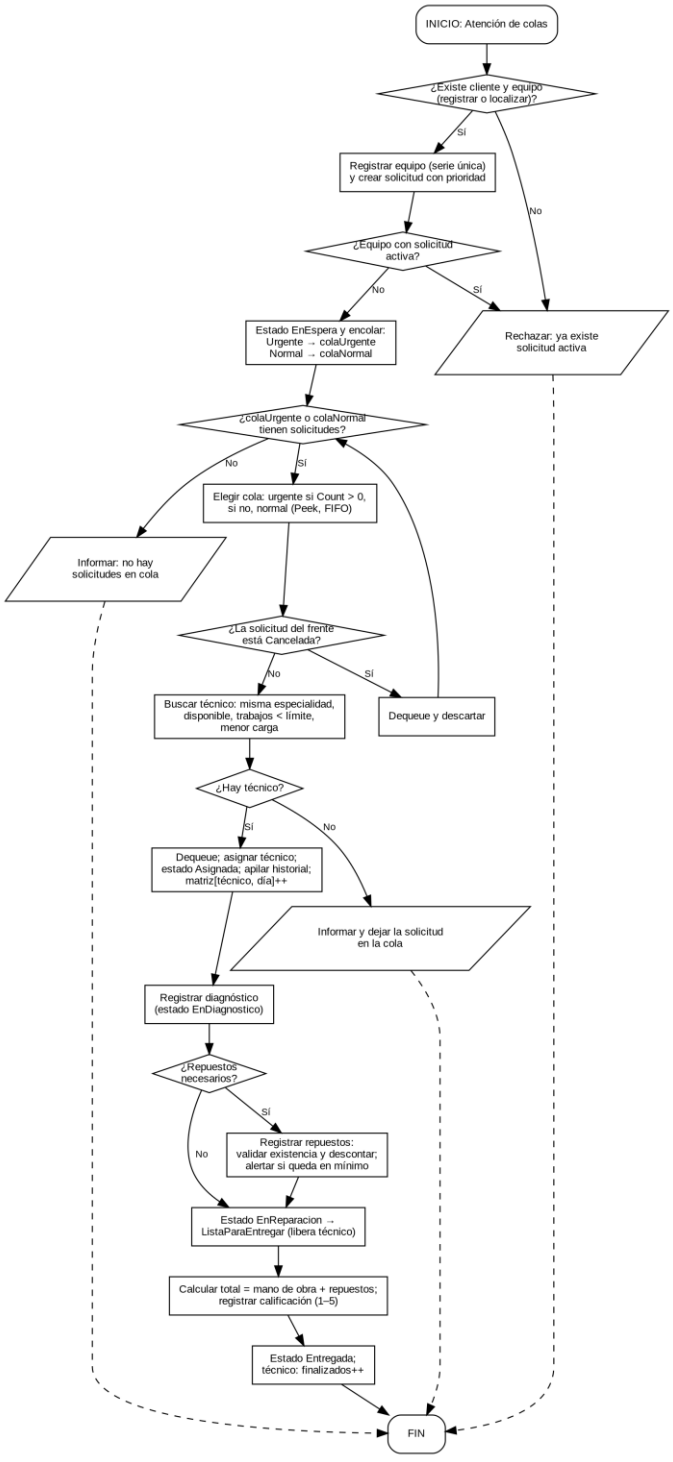
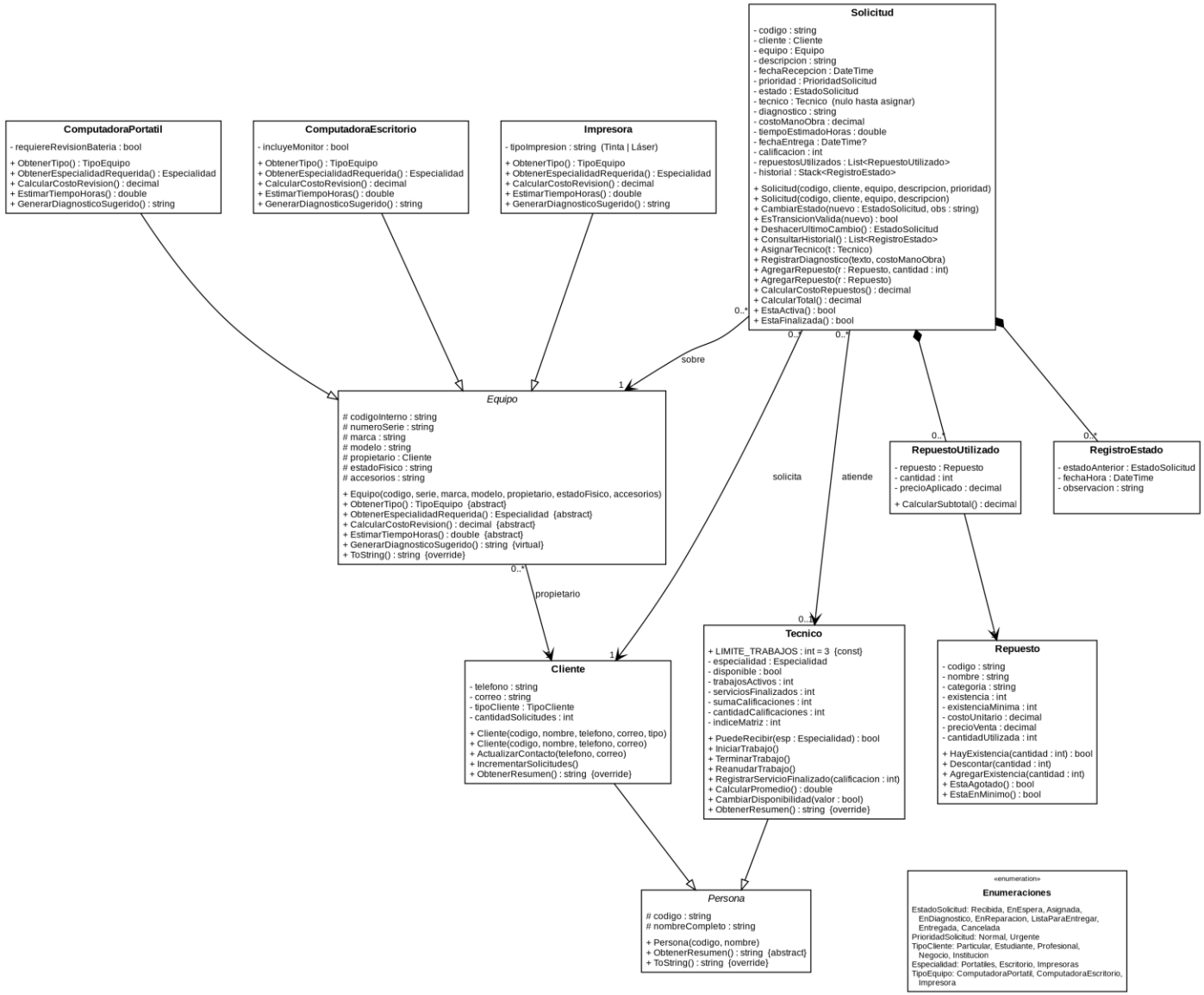


Figura 2. Flujo general de atención de una solicitud, desde el registro hasta la entrega (resume los nueve pasos del enunciado).

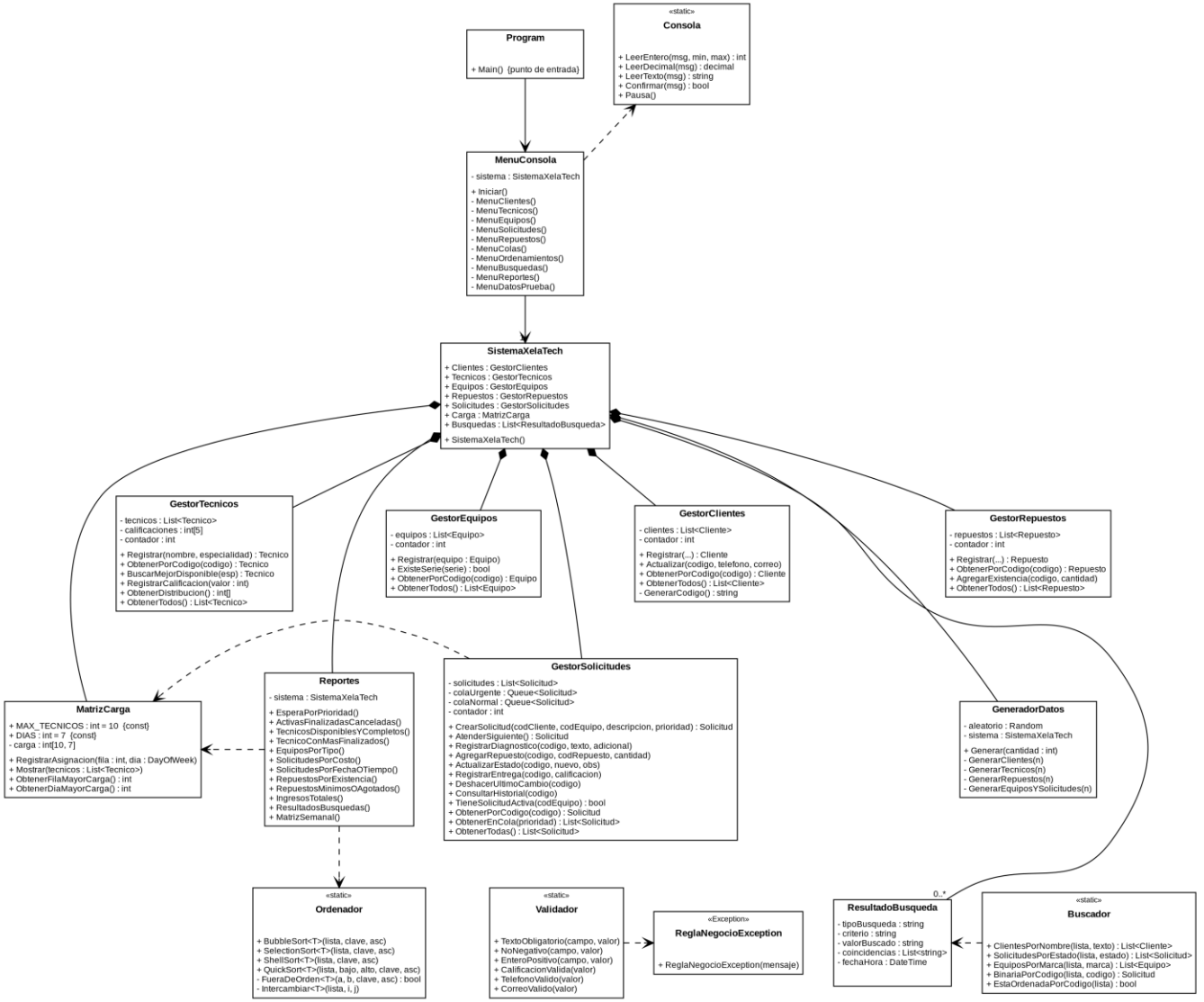


3. Diagrama de clases UML

Parte A: modelo de dominio. # protegido, - privado, + público. Triángulo vacío = herencia; flecha = asociación; rombo lleno = composición.



Parte B: servicios, control y algoritmos. Línea punteada = dependencia; rombo lleno = composición.



4. Uso de listas, colas, pilas y arreglos

Cada estructura participa directamente en las operaciones del sistema; ninguna se crea solo para demostrar sintaxis.

- **List<Cliente>** (en GestorClientes) — Contiene: Clientes registrados. Justificación: El número de clientes crece durante la ejecución y se consulta por posición y recorrido. Operaciones: Add, recorrido, búsqueda por código.
- **List<Tecnico>** (en GestorTecnicos) — Contiene: Técnicos. Justificación: Cantidad variable; se recorre para elegir el técnico apto y para reportes. Operaciones: Add, recorrido, selección del menos cargado.
- **List<Equipo>** (en GestorEquipos) — Contiene: Objetos de las tres subclases. Justificación: Una sola lista guarda tipos distintos gracias al polimorfismo. Operaciones: Add, validar serie, búsqueda por marca.
- **List<Solicitud>** (en GestorSolicitudes) — Contiene: Todas las solicitudes en orden de llegada. Justificación: Lista maestra: permite ordenar copias, buscar y generar reportes sin perder solicitudes. Operaciones: Add, recorrido, ordenar copia, búsqueda binaria.
- **List<Repuesto>** (en GestorRepuestos) — Contiene: Inventario. Justificación: Crece al registrar repuestos; se ordena por existencia. Operaciones: Add, descuento, Shell Sort.
- **List<RepuestoUtilizado>** (en Dentro de cada Solicitud) — Contiene: Repuestos consumidos y precio aplicado. Justificación: Cantidad variable por solicitud; base del costo de repuestos. Operaciones: Add, suma de subtotales.
- **List<ResultadoBusqueda>** (en SistemaXelaTech) — Contiene: Búsquedas ejecutadas. Justificación: Permite mostrar en el reporte 11 los resultados de las búsquedas. Operaciones: Add, recorrido.
- **Queue<Solicitud> colaUrgente** (en GestorSolicitudes) — Contiene: Solicitudes urgentes en espera. Justificación: El acceso es FIFO: se atiende primero la que llegó antes. Operaciones: Enqueue, Peek, Dequeue, Count.
- **Queue<Solicitud> colaNormal** (en GestorSolicitudes) — Contiene: Solicitudes normales en espera. Justificación: Igual que la anterior; solo se usa cuando la urgente está vacía. Operaciones: Enqueue, Peek, Dequeue, Count.
- **Stack<RegistroEstado> historial** (en Dentro de cada Solicitud) — Contiene: Estados anteriores con fecha y observación. Justificación: El último cambio es el primero en deshacerse (LIFO). Operaciones: Push al cambiar estado; Count > 0 antes de Pop; recorrido para consultar.
- **int[5] calificaciones** (en GestorTecnicos) — Contiene: Cantidad de calificaciones 1 a 5 (índice = calificación − 1). Justificación: Tamaño fijo y conocido; contabiliza la distribución de calificaciones. Operaciones: Incremento al entregar; mostrar distribución.
- **int[3] equiposPorTipo** (en Reporte 5) — Contiene: Equipos atendidos por tipo (índice = TipoEquipo). Justificación: Tamaño fijo según los tres tipos de equipo. Operaciones: Conteo al generar el reporte.
- **int[10, 7] carga** (en MatrizCarga) — Contiene: Solicitudes asignadas por técnico (fila) y día (columna). Justificación: Representa una tabla de dos dimensiones fija; permite hallar técnico y día de mayor carga. Operaciones: Incremento al asignar; sumas por fila y columna.
- **Random** (en GeneradorDatos) — Contiene: —. Justificación: Produce prioridad, costos, tiempos y cantidades controlados. Operaciones: Next(min, max).

Detalle de la matriz de carga semanal

- Fila = `indiceMatriz` del técnico (se asigna al registrarlo, de 0 a 9). Columna = día de la semana (0 = lunes … 6 = domingo).
- Al asignar una solicitud: `carga[tecnico.indiceMatriz, díaDeLaFechaDeAsignación]++`.
- **Técnico con mayor carga:** fila con mayor suma. **Día con mayor carga:** columna con mayor suma. En caso de empate se muestra el primero encontrado.
- La matriz es un acumulado de asignaciones; no disminuye al cancelar o finalizar, porque representa el trabajo que se asignó.

5. Pseudocódigo de los ordenamientos y las búsquedas

Convenciones: ← asignación; = comparación; `lista[i]` elemento de la posición i (base 0); `clave(x)` devuelve el valor del criterio de ordenamiento del objeto x; `asc` es VERDADERO para orden ascendente y FALSO para descendente. Todos los algoritmos se programan manualmente; no se usa `Sort`, `OrderBy`, `ThenBy` ni `BinarySearch`.

5.1 Ordenamientos

- **Bubble Sort**
  Uso en el sistema: Reporte 6: solicitudes ordenadas por costo total
  Criterio (clave): `CalcularTotal()`
  Complejidad: O(n²)
- **Selection Sort**
  Uso en el sistema: Módulo 7: técnicos por cantidad de servicios finalizados
  Criterio (clave): `serviciosFinalizados`
  Complejidad: O(n²)
- **Shell Sort**
  Uso en el sistema: Reporte 8: repuestos por existencia disponible
  Criterio (clave): `existencia`
  Complejidad: Entre O(n log n) y O(n²), según la secuencia de brechas
- **Quick Sort**
  Uso en el sistema: Reporte 7: solicitudes por fecha de recepción o tiempo estimado; también por código antes de la búsqueda binaria
  Criterio (clave): `fechaRecepcion`, `tiempoEstimadoHoras` o `codigo`
  Complejidad: O(n log n) promedio; O(n²) peor caso

Funciones auxiliares comunes

```
FUNCIÓN FueraDeOrden(a, b, clave, asc) : booleano
    SI asc ENTONCES
        RETORNAR clave(a) > clave(b)
    SINO
        RETORNAR clave(a) < clave(b)

PROCEDIMIENTO Intercambiar(lista, i, j)
    temp ← lista[i]
    lista[i] ← lista[j]
    lista[j] ← temp

PROCEDIMIENTO OrdenarYReportar(origen, algoritmo, criterio, clave, asc)
    SI origen.Count < 2 ENTONCES
        LANZAR ReglaNegocioException("No hay datos suficientes para ordenar")
    copia ← NUEVA Lista con los elementos de origen        // el original conserva el orden de llegada
    EJECUTAR algoritmo sobre copia con (clave, asc)
    MOSTRAR "Algoritmo: ", algoritmo, " | Criterio: ", criterio, " | Orden: ", (asc ? "Ascendente" : "Descendente")
    PARA CADA elemento EN copia
        MOSTRAR fila del elemento
```

Bubble Sort

```
PROCEDIMIENTO BubbleSort(lista, clave, asc)
    n ← lista.Count
    PARA i ← 0 HASTA n - 2
        huboCambio ← FALSO
        PARA j ← 0 HASTA n - 2 - i
            SI FueraDeOrden(lista[j], lista[j + 1], clave, asc) ENTONCES
                Intercambiar(lista, j, j + 1)
                huboCambio ← VERDADERO
        SI NO huboCambio ENTONCES
            SALIR del ciclo externo                        // ya está ordenada
```

Selection Sort

```
PROCEDIMIENTO SelectionSort(lista, clave, asc)
    n ← lista.Count
    PARA i ← 0 HASTA n - 2
        indiceSel ← i                                      // posición del menor (asc) o mayor (desc)
        PARA j ← i + 1 HASTA n - 1
            SI FueraDeOrden(lista[indiceSel], lista[j], clave, asc) ENTONCES
                indiceSel ← j
```

```
    SI indiceSel ≠ i ENTONCES
      Intercambiar(lista, i, indiceSel)
```

Shell Sort

```
PROCEDIMIENTO ShellSort(lista, clave, asc)
  n ← lista.Count
  brecha ← n / 2                                  // división entera
  MIENTRAS brecha > 0
    PARA i ← brecha HASTA n - 1
      temp ← lista[i]
      j ← i
      MIENTRAS j ≥ brecha Y FueraDeOrden(lista[j - brecha], temp, clave, asc)
        lista[j] ← lista[j - brecha]
        j ← j - brecha
      lista[j] ← temp
    brecha ← brecha / 2
```

Quick Sort (partición de Lomuto, pivote = último elemento)

```
PROCEDIMIENTO QuickSort(lista, bajo, alto, clave, asc)
  SI bajo < alto ENTONCES
    p ← Particionar(lista, bajo, alto, clave, asc)
    QuickSort(lista, bajo, p - 1, clave, asc)
    QuickSort(lista, p + 1, alto, clave, asc)

FUNCIÓN Particionar(lista, bajo, alto, clave, asc) : entero
  pivote ← lista[alto]
  i ← bajo - 1
  PARA j ← bajo HASTA alto - 1
    SI NO FueraDeOrden(lista[j], pivote, clave, asc) ENTONCES    // lista[j] va antes o igual que el pivote
      i ← i + 1
      Intercambiar(lista, i, j)
  Intercambiar(lista, i + 1, alto)
  RETORNAR i + 1

// Llamada inicial: QuickSort(copia, 0, copia.Count - 1, clave, asc)
```

5.2 Búsquedas

Se diseñan **tres búsquedas secuenciales** (clientes por nombre parcial, solicitudes por estado y equipos por marca) y **una búsqueda binaria** (solicitud por código). Las cuatro operan sobre información real del sistema y registran su resultado para el reporte 11.

Búsqueda secuencial 1: clientes por nombre parcial

```
FUNCIÓN BuscarClientesPorNombre(clientes, texto) : Lista<Cliente>
  SI texto está vacío ENTONCES LANZAR ReglaNegocioException("Ingrese un texto a buscar")
  buscado ← MINÚSCULAS(RECORTAR(texto))
  coincidencias ← NUEVA Lista
  PARA CADA c EN clientes
    SI CONTIENE(MINÚSCULAS(c.NombreCompleto), buscado) ENTONCES
      coincidencias.Agregar(c)                    // se muestran TODAS las coincidencias
  GUARDAR ResultadoBusqueda("Secuencial", "Cliente por nombre parcial", texto, coincidencias)
  RETORNAR coincidencias
```

Búsqueda secuencial 2: solicitudes por estado

```
FUNCIÓN BuscarSolicitudesPorEstado(solicitudes, estadoBuscado) : Lista<Solicitud>
  coincidencias ← NUEVA Lista
  PARA CADA s EN solicitudes
    SI s.Estado = estadoBuscado ENTONCES
      coincidencias.Agregar(s)
  GUARDAR ResultadoBusqueda("Secuencial", "Solicitud por estado", estadoBuscado, coincidencias)
  RETORNAR coincidencias
```

Búsqueda secuencial 3: equipos por marca

```
FUNCIÓN BuscarEquiposPorMarca(equipos, marca) : Lista<Equipo>
  SI marca está vacía ENTONCES LANZAR ReglaNegocioException("Ingrese una marca")
  buscada ← MINÚSCULAS(RECORTAR(marca))
  coincidencias ← NUEVA Lista
  PARA CADA e EN equipos
    SI CONTIENE(MINÚSCULAS(e.Marca), buscada) ENTONCES
      coincidencias.Agregar(e)
  GUARDAR ResultadoBusqueda("Secuencial", "Equipo por marca", marca, coincidencias)
  RETORNAR coincidencias
```

Búsqueda binaria: solicitud por código

```
FUNCIÓN EstaOrdenadaPorCodigo(solicitudes) : booleano
   PARA i ← 1 HASTA solicitudes.Count - 1
      SI COMPARAR_ORDINAL(solicitudes[i - 1].Codigo, solicitudes[i].Codigo) > 0 ENTONCES
         RETORNAR FALSO
   RETORNAR VERDADERO

FUNCIÓN BusquedaBinariaPorCodigo(solicitudes, codigo) : Solicitud
   SI solicitudes.Count = 0 ENTONCES LANZAR ReglaNegocioException("No hay solicitudes registradas")
   SI NO EstaOrdenadaPorCodigo(solicitudes) ENTONCES
      LANZAR ReglaNegocioException("Los datos no están ordenados por código; ordénelos primero")
   izq ← 0
   der ← solicitudes.Count - 1
   MIENTRAS izq ≤ der
      medio ← izq + (der - izq) / 2
      cmp ← COMPARAR_ORDINAL(solicitudes[medio].Codigo, codigo)
      SI cmp = 0 ENTONCES
         GUARDAR ResultadoBusqueda("Binaria", "Solicitud por código", codigo, [solicitudes[medio]])
         RETORNAR solicitudes[medio]
      SINO SI cmp < 0 ENTONCES
         izq ← medio + 1
      SINO
         der ← medio - 1
   GUARDAR ResultadoBusqueda("Binaria", "Solicitud por código", codigo, [])
   RETORNAR NULO

// En el menú: si EstaOrdenadaPorCodigo es FALSO, se ofrece ordenar una copia con
// QuickSort(copia, 0, copia.Count - 1, codigo, VERDADERO) y buscar sobre esa copia.
```

6. Reglas del proceso y validaciones

6.1 Reglas del proceso

- **R1.** Cada registro recibe un código único generado por el sistema (prefijo + contador de 4 dígitos). Los códigos no se reutilizan.
- **R2.** Una solicitud nace en estado Recibida y pasa a EnEspera en el momento de encolarse.
- **R3.** Prioridad Urgente → `colaUrgente`; prioridad Normal → `colaNormal`.
- **R4.** Al atender, se toma de `colaUrgente` si tiene solicitudes; si no, de `colaNormal`. Dentro de cada cola el orden es FIFO.
- **R5.** La solicitud solo sale de la cola (Dequeue) si existe un técnico apto; de lo contrario permanece en su lugar y se informa.
- **R6.** Técnico apto: tiene la especialidad requerida por el equipo, está disponible y tiene menos de 3 trabajos activos. Se elige el de menor carga; en empate, el primero registrado.
- **R7.** Al asignar: estado Asignada, técnico con trabajosActivos + 1 y matriz[técnico, día] + 1. Al pasar a ListaParaEntregar o cancelarse, trabajosActivos − 1.
- **R8.** Los repuestos solo se registran con la solicitud en EnDiagnostico o EnReparacion. Se descuentan de inmediato y el precio de venta aplicado queda fijo.
- **R9.** Mano de obra = costo de revisión del equipo + costo adicional (≥ 0). Total = mano de obra + repuestos.
- **R10.** La entrega exige estado ListaParaEntregar y una calificación de 1 a 5. Registra la fecha de entrega, suma un servicio finalizado al técnico y actualiza el arreglo de calificaciones.
- **R11.** Cancelar es posible desde cualquier estado antes de Entregada. Libera al técnico si estaba asignado. Los repuestos ya descontados no se reintegran.
- **R12.** Deshacer solo se permite si la solicitud está en EnDiagnostico, EnReparacion o ListaParaEntregar. Si regresa desde ListaParaEntregar, el técnico retoma el trabajo (se valida su límite).
- **R13.** Los ordenamientos se hacen sobre copias y exigen al menos 2 datos. Cada reporte indica algoritmo, criterio y sentido.
- **R14.** La búsqueda binaria solo se ejecuta si la lista está ordenada por código; si no, se ofrece ordenarla (copia con Quick Sort).
- **R15.** Al crear una solicitud se incrementa `cantidadSolicitudes` del cliente.
- **R16.** Los datos de prueba pasan por los mismos métodos y validaciones, y comparten los contadores de código.

6.2 Transiciones de estado permitidas

- **Recibida** — EnEspera, Cancelada. **Observación:** El paso a EnEspera es automático al encolar.
- **EnEspera** — Asignada, Cancelada. **Observación:** Asignada solo ocurre desde Atención de colas.
- **Asignada** — EnDiagnostico, Cancelada. **Observación:** —
- **EnDiagnostico** — EnReparacion, Cancelada. **Observación:** Aquí se registran diagnóstico y repuestos.
- **EnReparacion** — ListaParaEntregar, Cancelada. **Observación:** Libera la carga activa del técnico al pasar a ListaParaEntregar.
- **ListaParaEntregar** — Entregada, Cancelada. **Observación:** Entregada requiere calificación.
- **Entregada** — Ninguno. **Observación:** Estado final: no se modifica ni se cancela.
- **Cancelada** — Ninguno. **Observación:** Estado final: no se deshace ni se modifica.

Cualquier otro cambio produce `ReglaNegocioException` (por ejemplo, pasar de EnDiagnostico a Entregada).

6.3 Validaciones obligatorias

- **V1.** Registrar códigos o números de serie duplicados. Cómo se valida: Códigos generados por contador; `GestorEquipos.ExisteSerie` antes de registrar.
- **V2.** Guardar campos obligatorios vacíos. Cómo se valida: `Validador.TextoObligatorio` (nulo, vacío o solo espacios).
- **V3.** Costos, existencias, cantidades o calificaciones negativas. Cómo se valida: `Validador.NoNegativo`, `EnteroPositivo` y `CalificacionValida`.
- **V4.** Crear una solicitud para cliente o equipo inexistente. Cómo se valida: `ObtenerPorCodigo` devuelve nulo → excepción.
- **V5.** Más de una solicitud activa para el mismo equipo. Cómo se valida: `TieneSolicitudActiva(codEquipo)` al crear.
- **V6.** Asignar un técnico sin la especialidad requerida. Cómo se valida: `Tecnico.PuedeRecibir(equipo.ObtenerEspecialidadRequerida())`.
- **V7.** Superar el límite de trabajos activos. Cómo se valida: `trabajosActivos < LIMITE_TRABAJOS` al asignar o reanudar.
- **V8.** Usar repuestos sin existencia suficiente. Cómo se valida: `Repuesto.HayExistencia(cantidad)` antes de descontar.
- **V9.** Cambiar estados en orden incorrecto. Cómo se valida: `Solicitud.EsTransicionValida` (tabla 8.2).
- **V10.** Desapilar con historial vacío. Cómo se valida: Verificar `historial.Count > 0` antes de `Pop`.
- **V11.** Búsqueda binaria sobre datos no ordenados. Cómo se valida: `Buscador.EstaOrdenadaPorCodigo` antes de buscar.
- **V12.** Modificar o cancelar una solicitud entregada. Cómo se valida: Todo método que modifica revisa que el estado no sea Entregada ni Cancelada.

.4 Validaciones adicionales del diseño

- **VA1.** Teléfono con exactamente 8 dígitos numéricos. Cómo se valida: `Validador.TelefonoValido`.
- **VA2.** Correo con formato válido y no repetido entre clientes. Cómo se valida: `Validador.CorreoValido` y recorrido de `GestorClientes`.
- **VA3.** Calificación solo entre 1 y 5, y solo al entregar. Cómo se valida: Rango en `RegistrarEntrega`.
- **VA4.** Precio de venta mayor o igual al costo unitario; existencia mínima ≥ 0. Cómo se valida: Validación al registrar el repuesto.
- **VA5.** No registrar diagnóstico vacío ni sin técnico asignado. Cómo se valida: Revisión en `RegistrarDiagnostico`.
- **VA6.** Repuestos solo en EnDiagnostico o EnReparacion, con cantidad mayor que cero. Cómo se valida: Revisión en `AgregarRepuesto`.
- **VA7.** No registrar más de 10 técnicos (filas de la matriz). Cómo se valida: Revisión en `GestorTecnicos.Registrar`.
- **VA8.** No asignar técnicos marcados como no disponibles. Cómo se valida: `PuedeRecibir` revisa `disponible`.
- **VA9.** No deshacer un cambio si el técnico excedería su límite. Cómo se valida: `ReanudarTrabajo` valida el límite.
- **VA10.** No atender colas vacías ni ordenar con menos de 2 datos. Cómo se valida: `Count` de colas y listas antes de procesar.

- **VA11.** El propietario del equipo debe ser un cliente existente. Cómo se valida: `GestorClientes.ObtenerPorCodigo` al registrar el equipo.
- **VA12.** Opciones de menú y datos numéricos dentro de rango. Cómo se valida: `Consola.LeerEntero` con `TryParse`; el menú nunca se cierra por un error.

8.5 Manejo de excepciones

- Las reglas del negocio incumplidas lanzan `ReglaNegocioException` con un mensaje claro.
- `MenuConsola` captura la excepción en el ciclo de cada submenú, muestra el mensaje y regresa al menú sin cerrar el programa.
- Las entradas numéricas se leen con `TryParse` y se repiten hasta ser válidas; un error inesperado se captura con `Exception` general.

7. Diseño preliminar del menú y de los reportes

7.1 Menú principal

```
==========================================
          XELATECH SOPORTE
==========================================
 1. Gestión de clientes
 2. Gestión de técnicos
 3. Gestión de equipos
 4. Gestión de solicitudes
 5. Gestión de repuestos
 6. Atención de colas
 7. Ordenamientos
 8. Búsquedas
 9. Reportes
10. Generar datos de prueba
11. Salir
------------------------------------------
Seleccione una opción (1-11): _
```

7.2 Submenús

- **1. Clientes:** 1. Registrar cliente · 2. Consultar clientes · 3. Actualizar cliente · 4. Volver
- **2. Técnicos:** 1. Registrar técnico · 2. Consultar técnicos · 3. Consultar por especialidad · 4. Cambiar disponibilidad · 5. Distribución de calificaciones · 6. Volver
- **3. Equipos:** 1. Registrar computadora portátil · 2. Registrar computadora de escritorio · 3. Registrar impresora · 4. Consultar equipos · 5. Volver
- **4. Solicitudes:** 1. Crear solicitud (normal o urgente) · 2. Registrar diagnóstico · 3. Registrar repuestos · 4. Actualizar estado (incluye cancelar) · 5. Registrar entrega · 6. Consultar historial de estados · 7. Deshacer último cambio · 8. Consultar solicitudes · 9. Volver
- **5. Repuestos:** 1. Registrar repuesto · 2. Consultar repuestos · 3. Agregar existencia · 4. Volver
- **6. Atención de colas:** 1. Ver colas · 2. Atender siguiente solicitud · 3. Mostrar matriz de carga semanal · 4. Volver
- **7. Ordenamientos:** 1. Solicitudes por costo total (Bubble Sort) · 2. Técnicos por servicios finalizados (Selection Sort) · 3. Repuestos por existencia (Shell Sort) · 4. Solicitudes por fecha o tiempo estimado (Quick Sort) · 5. Volver. Cada opción pregunta ascendente o descendente.
- **8. Búsquedas:** 1. Clientes por nombre parcial · 2. Solicitudes por estado · 3. Equipos por marca · 4. Solicitud por código (binaria) · 5. Volver
- **9. Reportes:** Reportes 1 a 12 (sección 9.3) · 13. Volver
- **10. Datos de prueba:** Solicita la cantidad, confirma y genera los datos con Random.

7.3 Reportes

Todos los reportes se construyen recorriendo los datos almacenados durante la ejecución; ningún resultado se escribe directamente en el código. Si no hay datos, el reporte lo indica.

- **1. Solicitudes en espera por prioridad** — Fuente: `colaUrgente`, `colaNormal`. Contenido: Código, cliente, equipo, fecha de recepción. Proceso: Recorre cada cola sin desencolar; dos bloques (urgentes y normales) con total.
- **2. Activas, finalizadas y canceladas** — Fuente: `List<Solicitud>`. Contenido: Código, cliente, estado, total. Proceso: Clasifica con `EstaActiva`/`EstaFinalizada`; muestra tres bloques y conteos.
- **3. Técnicos disponibles y con carga completa** — Fuente: `List<Tecnico>`. Contenido: Código, nombre, especialidad, activos / límite. Proceso: Disponible: indicador activo y activos < 3. Carga completa: activos = 3.
- **4. Técnico con más servicios finalizados** — Fuente: `List<Tecnico>`. Contenido: Código, nombre, finalizados, promedio. Proceso: Recorrido que busca el máximo; informa empates.
- **5. Equipos atendidos por tipo** — Fuente: `int[3]` equiposPorTipo. Contenido: Tipo, cantidad. Proceso: Cuenta solicitudes asignadas o posteriores (no canceladas) por tipo de equipo.
- **6. Solicitudes ordenadas por costo total** — Fuente: Copia de `List<Solicitud>`. Contenido: Código, cliente, mano de obra, repuestos, total. Proceso: **Bubble Sort**; indica criterio y sentido.
- **7. Solicitudes por fecha o tiempo estimado** — Fuente: Copia de `List<Solicitud>`. Contenido: Código, fecha, tiempo estimado, prioridad. Proceso: **Quick Sort**; el usuario elige criterio y sentido.
- **8. Repuestos ordenados por existencia** — Fuente: Copia de `List<Repuesto>`. Contenido: Código, nombre, categoría, existencia, mínima. Proceso: **Shell Sort**; indica criterio y sentido.
- **9. Repuestos con existencia mínima o agotados** — Fuente: `List<Repuesto>`. Contenido: Código, nombre, existencia, mínima, situación. Proceso: Existencia ≤ mínima → MÍNIMO; existencia = 0 → AGOTADO.
- **10. Ingresos totales por servicios finalizados** — Fuente: `List<Solicitud>`. Contenido: Código, total y suma general. Proceso: Suma el total de solicitudes Entregadas; muestra cantidad y promedio.
- **11. Resultados de las búsquedas** — Fuente: `List<ResultadoBusqueda>`. Contenido: Tipo, criterio, valor buscado, coincidencias. Proceso: Muestra la última ejecución de cada búsqueda; indica las no ejecutadas.
- **12. Matriz semanal de carga de trabajo** — Fuente: `int[10, 7]`. Contenido: Técnicos × días, totales, mayor carga. Proceso: Muestra la matriz e identifica técnico y día con mayor carga.

7.4 Ejemplos de diseño de salida

Reporte 6

```
REPORTE: SOLICITUDES ORDENADAS POR COSTO TOTAL
Algoritmo: Bubble Sort | Criterio: Costo total | Orden: Descendente
--------------------------------------------------------------------------------
No.  Código       Cliente          Mano obra   Repuestos     Total
 1   SOL-0007     María López        Q 210.00    Q 350.00    Q 560.00
 2   SOL-0003     Café El Portal     Q  90.00    Q 120.00    Q 210.00
--------------------------------------------------------------------------------
Solicitudes ordenadas: 2
```

Reporte 12

```
MATRIZ SEMANAL DE CARGA DE TRABAJO (solicitudes asignadas)
Técnico        Lun  Mar  Mié  Jue  Vie  Sáb  Dom | Total
TEC-0001         1    0    2    0    1    0    0 |   4
TEC-0002         0    1    1    0    0    1    0 |   3
TEC-0003         0    1    0    1    0    0    0 |   2
Total por día    1    2    3    1    1    1    0 |   9
-----------------------------------------------------------------
Técnico con mayor carga: TEC-0001 (4 solicitudes)
Día con mayor carga:     Miércoles (3 solicitudes)
```